درس الفصل الثاني عشر

القوائم المتصلة

العقد، الإضافة، الحذف، والمرور على القائمة

الأستاذ: د. بن عبد الرزاق يوسف

الفئة: طلبة السنة الأولى

أهداف الدرس

في نهاية هذا الدرس، تستطيع أنْ:

- تعرّف عقدة القائمة المتصلة بهيكل ومؤشر.
- تضيف عقدة في بداية القائمة وفي نهايتها، وتمر على عناصرها.
- تبحث عن قيمة، وتحذف عقدة، وتحرر القائمة.

القائمة المتصلة

تعريف

القائمة المتصلة: سلسلة من **العقد**، كل عقدة فيها قيمة ومؤشر نحو العقدة التالية.

لماذا القوائم المتصلة؟

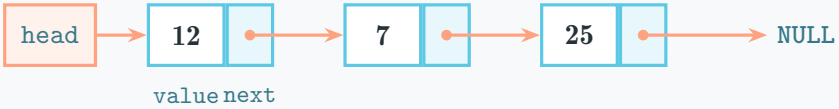
القائمة المتصلة

تكبر وتصغر أثناء التنفيذ،
والإضافة تغيّر مؤشرات فقط.

الجدول

حجمه ثابت، والإضافة في
الوسط تتطلب إزاحة العناصر.

القائمة في الذاكرة



كل عقدة تحتوي على قيمة وعلى مؤشر نحو العقدة التالية. آخر عقدة تشير إلى NULL.

تعريف العقدة

```c
typedef struct Node {
    int value;
    struct Node *next;
} Node;
```

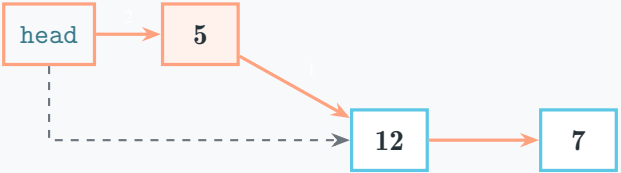
الحقل `next` مؤشر نحو هيكل من النوع نفسه.

إنشاء عقدة

```c
Node *newNode(int v) {
    Node *n = malloc(sizeof(Node));
    if (n == NULL) exit(1);
    n->value = v;
    n->next = NULL;
    return n;
}
```

نحجز كل عقدة بـmalloc، ونحتاج إلى stdlib.h.

الإضافة في البداية: خطوة بخطوة



1) العقدة الجديدة تشير إلى البداية القديمة. 2) `head` يشير إلى العقدة الجديدة.

الإضافة في البداية والعرض

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *next;
} Node;

Node *push(Node *head, int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v;
    n->next = head;      /* step 1 */
    return n;            /* step 2 */
}
```

```c
int main(void) {
    Node *head = NULL, *p;
    head = push(head, 7);
    head = push(head, 12);
    head = push(head, 5);
    for (p = head; p; p = p->next)
        printf("%d -> ", p->value);
    printf("NULL\n");
    return 0;
}
```

```
5 -> 12 -> 7 -> NULL
```

نمر على القائمة بمؤشر p حتى NULL.

الإضافة في النهاية

```c
Node *append(Node *head, int v) {
    Node *n = newNode(v), *p = head;
    if (head == NULL)
        return n;
    while (p->next != NULL)     /* go to the last node */
        p = p->next;
    p->next = n;
    return head;
}
```

نصل إلى آخر عقدة، ثم نربطها بالعقدة الجديدة.

الطول والمجموع

```c
int length(Node *head) {
    int n = 0;
    while (head != NULL) {
        n++;
        head = head->next;
    }
    return n;
}
```

المعالجة نفسها تحسب المجموع أو أكبر قيمة.

البحث عن قيمة

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *next;
} Node;

Node *push(Node *h, int v) {
    Node *n = malloc(sizeof(Node));
    n->value = v; n->next = h;
    return n;
}
Node *find(Node *p, int x) {
    while (p && p->value != x)
        p = p->next;
    return p;       /* NULL: not found */
}
```

```c
int main(void) {
    Node *h = push(push(NULL, 4), 9), *t;
    if (find(h, 9))
        printf("9 found\n");
    if (find(h, 8) == NULL)
        printf("8 not found\n");
    while (h) {
        t = h; h = h->next; free(t);
    }
    return 0;
}
```

```
9 found
8 not found
```

`find` تعيد `NULL` إن لم تجد القيمة.

حذف العقدة الأولى

```
Node *popFront(Node *head) {
    Node *old = head;
    if (head == NULL)
        return NULL;
    head = head->next;
    free(old);
    return head;
}
```

نحفظ العقدة القديمة قبل تحريك head، ثم نحررها.

حذف قيمة

```c
Node *removeValue(Node *head, int x) {
    Node *p = head, *prev = NULL;
    while (p != NULL && p->value != x) {
        prev = p;
        p = p->next;
    }
    if (p == NULL) return head;          /* not found */
    if (prev == NULL) head = p->next;    /* first node */
    else prev->next = p->next;           /* skip p */
    free(p);
    return head;
}
```

نربط العقدة السابقة بالعقدة التالية، فنتجاوز العقدة المحذوفة.

تحرير القائمة

```c
void freeList(Node *head) {
    Node *t;
    while (head != NULL) {
        t = head;
        head = head->next;
        free(t);
    }
}
```

كل `malloc` يقابله `free`.

الجدول أم القائمة المتصلة؟

	الجدول	القائمة المتصلة
الحجم	ثابت	متغير أثناء التنفيذ
الوصول إلى العنصر `i`	مباشر: `T[i]`	نمر على العقد واحدة واحدة
الإضافة في البداية	إزاحة كل العناصر	تغيير مؤشرين

أمثلة تطبيقية

عشرة أمثلة من مجالات مختلفة

نطبق ما تعلمناه في مجالات من الحياة اليومية: الصحة، الرياضة، الطقس، النقل، التجارة الإلكترونية، ألعاب الفيديو، الشبكات الاجتماعية، الهواتف الذكية، البيئة، الطاقة.

المثال الأول: الصحة

نبضات القلب: نبني القائمة من جدول، ثم نمر على العقد لحساب المعدل.

```
Average = 77 bpm
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[5] = {72, 80, 75, 90, 68}, i, sum = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 5; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        sum += n->value;
    printf("Average = %d bpm\n", sum / 5);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

المثال الثاني: الرياضة

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[6] = {1, 3, 0, 2, 4, 1}, i, count = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 6; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        if (n->value >= 2) count++;
    printf("Matches with 2+ goals: %d\n", count);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

مباريات بهدفين أو أكثر: نعدّ العقد التي قيمتها 2 أو أكثر.

```
Matches with 2+ goals: 3
```

المثال الثالث: الطقس

أعلى درجة حرارة: نفترض أن العقدة الأولى هي الأعلى، ثم نقارن بالباقي.

```
Max = 35 C
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[5] = {24, 31, 28, 35, 29}, i, max;
    Node *head = NULL, *n;
    for (i = 0; i < 5; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    max = head->value;
    for (n = head; n != NULL; n = n->next)
        if (n->value > max) max = n->value;
    printf("Max = %d C\n", max);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

المثال الرابع: النقل

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[3] = {101, 102, 103}, i;
    Node *head = NULL, *n;
    for (i = 0; i < 3; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        printf("%d -> ", n->value);
    printf("NULL\n");
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

محطات الحافلة: نعرض أرقام المحطات. الإضافة في البداية تعكس الترتيب.

```
103 -> 102 -> 101 -> NULL
```

المثال الخامس: التجارة الإلكترونية

مبلغ السلة: كل عقدة سعر منتج، ونجمع الأسعار.

```
Total = 7690 DA
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[3] = {2500, 890, 4300}, i, total = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 3; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        total += n->value;
    printf("Total = %d DA\n", total);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

المثال السادس: ألعاب الفيديو

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[3] = {75, 100, 60}, i, found = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 3; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        if (n->value == 100) found = 1;
    if (found) printf("100 found\n");
    else printf("Not found\n");
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

البحث عن نتيجة: هل حقق أحد اللاعبين 100 نقطة؟

```
100 found
```

المثال السابع: الشبكات الإجتماعية

المنشورات النابجة: نعدّ المنشورات التي تجاوزت 50 إعجاباً.

```
Popular posts: 3
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[5] = {12, 85, 40, 120, 66}, i, popular = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 5; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        if (n->value > 50) popular++;
    printf("Popular posts: %d\n", popular);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

المثال الثامن: الهواتف الذكية

عدد الإشعارات: نحسب طول القائمة: عدد الإشعارات غير المقروءة.

```
Unread: 4
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[4] = {7, 3, 9, 1}, i, count = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 4; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        count++;
    printf("Unread: %d\n", count);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

المثال التاسع: البيئة

أيام الهواء النقي: نعدّ الأيام التي كان فيها مؤشر جودة الهواء 50 أو أقل.

```
Clean-air days: 3
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[5] = {42, 85, 30, 60, 48}, i, clean = 0;
    Node *head = NULL, *n;
    for (i = 0; i < 5; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next)
        if (n->value <= 50) clean++;
    printf("Clean-air days: %d\n", clean);
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

المثال العاشر: الطاقة

تعديل القيم: نمر على العقد ونضيف 10 كيلوواط ساعي لكل قيمة، ثم نعرضها.

```
150 kWh
105 kWh
130 kWh
```

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int value; struct Node *next; } Node;
int main(void) {
    int t[3] = {120, 95, 140}, i;
    Node *head = NULL, *n;
    for (i = 0; i < 3; i++) {    /* build */
        n = malloc(sizeof(Node));
        n->value = t[i]; n->next = head;
        head = n;
    }
    for (n = head; n != NULL; n = n->next) {
        n->value = n->value + 10;
        printf("%d kWh\n", n->value);
    }
    while (head != NULL) {          /* free */
        n = head; head = head->next; free(n);
    }
    return 0;
}
```

أخطاء شائعة

تجنّب هذه الأخطاء

- فقدان `head`: تحريكه دون حفظ البداية يضيّع القائمة.
- نسيان `next = NULL` في آخر عقدة، أو استعمال `p->next` و`p` يساوي `NULL`.
- نسيان `free`، أو استعمال عقدة بعد تحريرها.

خلاصة الدرس

خلاصة الدرس

- القائمة المتصلة عقد مرتبطة بمؤشرات، تبدأ بـ`head` وتنتهي بـ`NULL`.
- الإضافة والحذف يغيّران المؤشرات فقط، دون إزاحة العناصر.
- نمر على القائمة بـ`p = p->next`، ونحرر كل العقد في النهاية.